HACK IN HILLS '26

CYBERSECURITY & AI DEFENSE TRACK

SafeArena

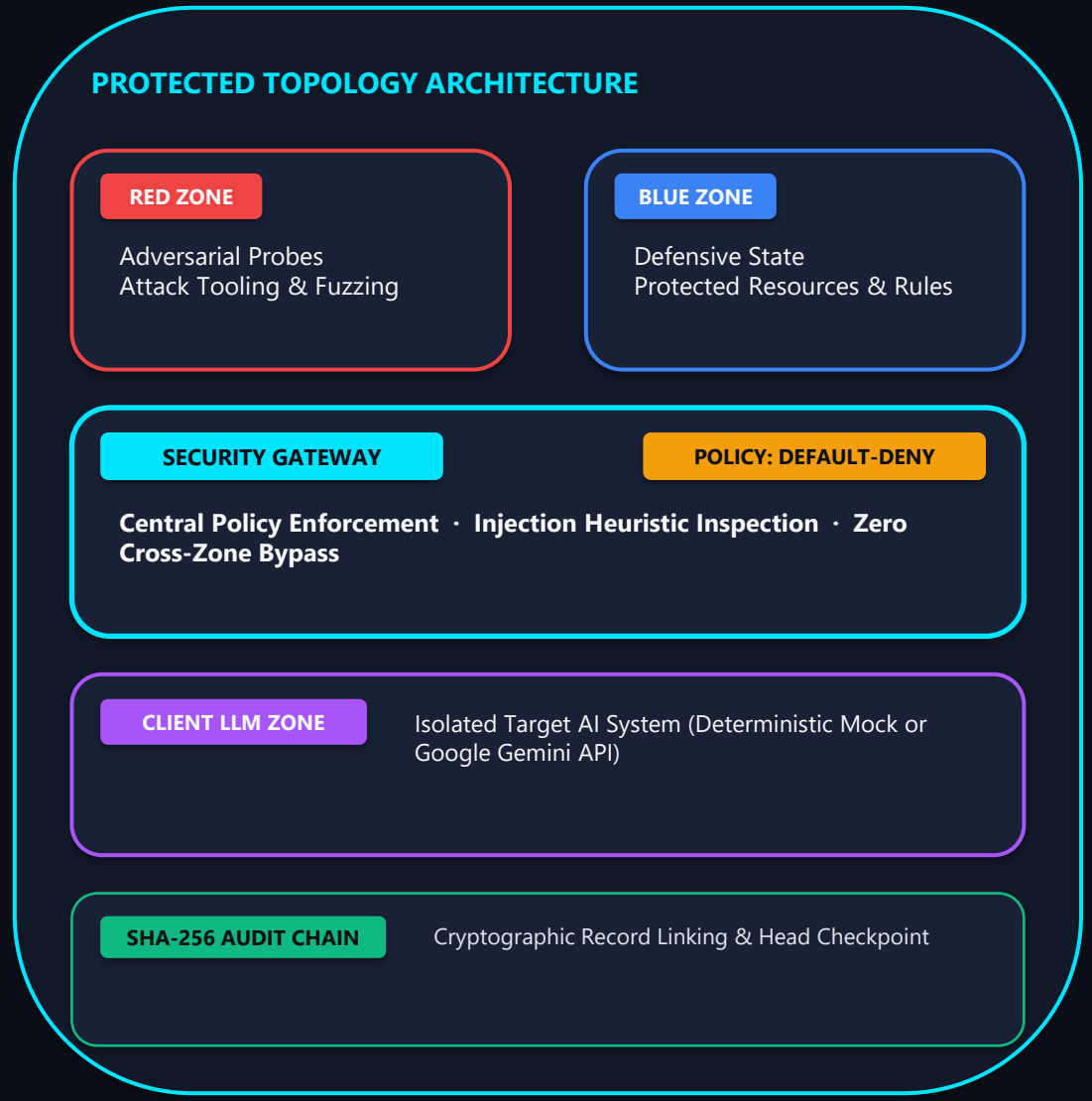
Isolate → Control → Detect → Prove

A Controlled Security Testing Environment for AI Systems

SafeArena enables authorized adversarial AI evaluation while preventing unauthorized cross-zone lateral movement. Every request is mediated by a default-deny security gateway, monitored for prompt-injection threats, and sealed in a tamper-evident SHA-256 audit ledger.

TEAM: VSV The Tech Giants

PROJECT STATUS: Functional Working Prototype (103 Unit & API Tests Passing)
DEPLOYMENT MODES: Local Policy Simulation | Multi-Container Docker Isolation



01 // THE THREAT LANDSCAPE

The Danger of Uncontrolled AI Security Testing

Evaluating AI systems in shared, unsegmented environments introduces severe security and operational liabilities.

RED TEAM RISK

Uncontained Adversarial Blast Radius

- Adversarial prompts, jailbreaks, and autonomous agent exploits are often tested on shared corporate networks.
- Without hard network and process isolation, payloads can execute lateral movement toward internal databases or defenses.
- Uncontrolled tests can inadvertently exfiltrate sensitive credentials, canary tokens, or proprietary prompts.
- **CRITICAL RISK: Testing red-team exploits can unintentionally compromise internal production state.**

ENFORCEMENT GAP

Direct, Unmediated Access Paths

- Testing tools routinely connect directly to target LLM endpoints without an intervening policy enforcement layer.
- There is no active policy inspection determining whether the caller is authorized to invoke specific model tools or modes.
- Cross-zone communications (such as red attackers probing blue defenses) proceed unhindered.
- **CRITICAL RISK: Security teams cannot enforce granular access boundaries between testers, models, and defended assets.**

EVIDENTIARY BLINDSPOT

Ephemeral & Malleable Audit Logs

- Test logs are frequently relegated to terminal stdout, ephemeral debug files, or mutable database tables.
- Standard log files can be arbitrarily edited, truncated, reordered, or deleted by system operators or attackers.
- Auditors, governance boards, and GRC teams have zero cryptographic assurance of what actually happened during testing.
- **CRITICAL RISK: Inability to prove non-repudiation or satisfy emerging AI compliance frameworks (NIST AI RMF, EU AI Act).**

CORE PROBLEM: AI security testing requires an environment where red teams can be aggressively adversarial, blue assets remain strictly untouchable, and every security event is cryptographically non-repudiable.

02 // THE DESIGN GAP

Why Existing Approaches Leave an Evidentiary Void

Current tooling treats testing, network isolation, policy enforcement, and audit logging as disconnected silos.

AD-HOC SCRIPTING

Uncontrolled / Raw Scripts

Environment: Local Python scripts, curl, or Jupyter

Zone Isolation: None: shared host OS & local network

Policy Control: None: direct socket connections to model

Event Visibility: Ephemeral stdout prints; lost on exit

Audit Integrity: Non-existent: raw logs are editable text

Defensibility: Zero proof of what actually occurred

FRAGMENTED ENTERPRISE

Generic Proxies + Heavy SIEMs

Environment: Complex cloud VPCs with generic proxies

Zone Isolation: Coarse-grained network security groups

Policy Control: Generic L7 HTTP rules; blind to AI threats

Event Visibility: Delayed batch ingestion into cloud SIEMs

Audit Integrity: Mutable SQL records; vulnerable to root

Defensibility: High cost; disconnected from AI context

SAFEARENA BLUEPRINT

Unified AI Security Fabric

Environment: Dedicated Red/Blue sandboxes + Gateway

Zone Isolation: Docker internal subnets + dropped caps

Policy Control: Default-deny gateway with AI inspection

Event Visibility: Immediate, structured, severity-rated stream

Audit Integrity: Tamper-evident SHA-256 hash chains

Defensibility: Mathematically verifiable audit proof

THE TAKEAWAY: SafeArena is not just a testing script or a generic firewall; it is an integrated, auditable security fabric.

03 // ARCHITECTURAL PILLARS

The SafeArena Framework: One Integrated Workflow

A closed-loop security architecture designed to isolate testing, control access, detect threats, and prove integrity.

1. ISOLATE

ZONE SEGREGATION

Physical & Logical Zone Containment

- Red Sandbox (adversarial testing) and Blue Sandbox (defensive state) exist in completely separate zones.
- In Docker mode, each zone is bound to an isolated bridge network (red_net, blue_net, llm_net).
- Networks configured with internal: true (no direct cross-zone routing and no public internet route).
- Container hardening: read-only root filesystems, dropped capabilities (cap_drop: ALL), unprivileged user.

2. CONTROL

SECURITY GATEWAY

Default-Deny Policy Enforcement

- The Security Gateway is the only entity attached to all zone networks; direct bypass is impossible.
- Strict default-deny policy: anything not explicitly authorized is blocked immediately.
- Authorized: Red → LLM (ALLOW), Blue → LLM (ALLOW).
- Unauthorized: Red → Blue (DENY 403), Blue → Red (DENY 403).
- Blocked requests are dropped at the gateway; payloads never touch the target destination.

3. DETECT

EVENT TELEMETRY

Severity-Rated Security Events

- Every decision produces a structured security event persisted to SQLite (events.db).
- Granular severity ratings: INFO (normal allowed calls), MEDIUM (injection heuristic hit), HIGH (policy violation), CRITICAL (canary leak).
- Regex heuristics detect adversarial prompt patterns and inspect model refusal states.
- Live operator console offers instant multi-parameter filtering and full-text search.

4. PROVE

SHA-256 AUDIT LEDGER

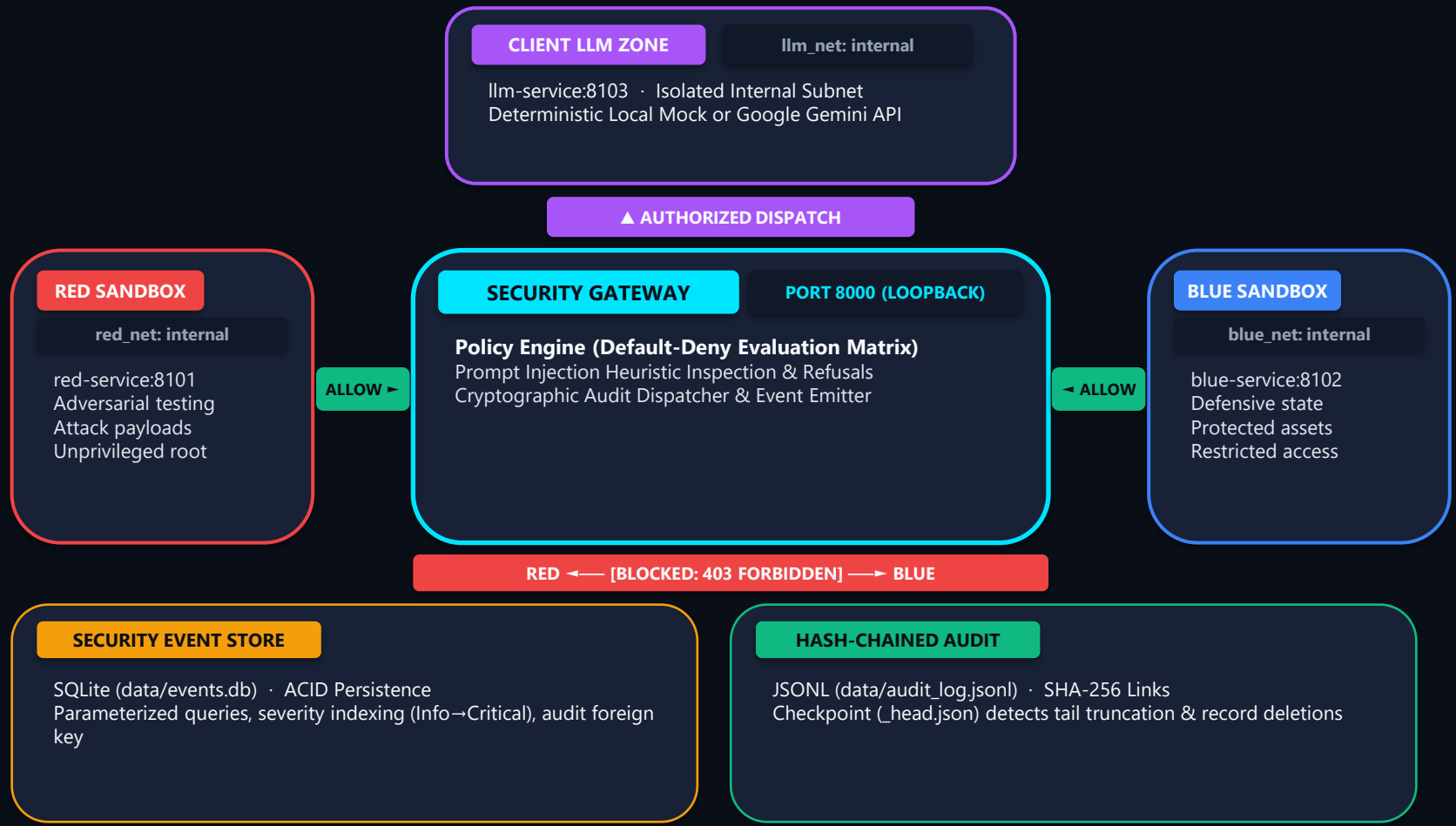
Tamper-Evident Hash Chaining

- Every request is cryptographically appended to an immutable JSONL audit ledger (audit_log.jsonl).
- Record N embeds the SHA-256 digest of Record N-1, creating an unbroken mathematical chain from GENESIS.
- A separate head checkpoint (_head.json) detects tail truncation and record deletions.
- Single-click verification recalculates every hash in milliseconds; proven against 7 corruption scenarios.

04 // DEFENSE-IN-DEPTH TOPOLOGY

SafeArena System Architecture & Gateway Mediation

Strict gateway mediation guarantees that adversarial testing is contained while unauthorized paths are blocked.



ACCESS POLICY MATRIX

Explicit Policy Rules:

Red → LLM: ALLOW (200)
└ Authorized adversarial probe via Gateway

Blue → LLM: ALLOW (200)
└ Authorized defensive prompt via Gateway

Red → Blue: DENY (403)
└ Blocked: Cannot touch defense state

Blue → Red: DENY (403)
└ Blocked: Cannot inspect attack payload

LLM → Red: DENY (403)
└ Blocked: Model cannot initiate outbound

Unknown: DENY (403)
└ Default-deny on unlisted zone / route

05 // THE REQUEST LIFECYCLE

How a Request Works: From Ingress to Cryptographic Proof

A rigorous 6-stage evaluation pipeline executed in milliseconds for every single test invocation.

1. INGRESS

Probe Ingress

- Source zone initiates request to Gateway API.
- JSON payload: {source, target, payload}.
- Gateway captures timestamp and generates unique request_id and session_id.

2. EVALUATE

Policy Check

- Policy engine normalizes source and target.
- Queries explicit access matrix.
- Default-deny rule: anything unlisted is marked unauthorized.
- Decision object formed.

3. ENFORCE

Gate Decision

- If DENIED (e.g. Red→Blue): Immediate HTTP 403 response.
- Payload is completely dropped; never forwarded.
- If ALLOWED (Red→LLM): Passed to inspection pipeline.

4. INSPECT

Heuristics & LLM

- Regex heuristic scans for prompt injection patterns.
- Payload forwarded to Client LLM (mock or Gemini).
- Model response, refusal, or canary leak captured.

5. CHAIN

SHA-256 Ledger

- Canonical JSON created (sorted keys, ASCII safe).
- hash = SHA256(canonical JSON + prev_hash).
- Appended to audit_log.jsonl.
- Atomic update to _head.json.

6. TELEMETRY

Events & UI

- Structured security event committed to SQLite.
- Event stores audit_hash foreign reference.
- Live UI updates: counters, event log, and audit verification table.

DUAL-LAYER CONTROL ARCHITECTURE: The Security Gateway policy decision is an application-level control (inspecting payload and metadata). Docker network segmentation provides a separate, physical network-level control (preventing raw socket bypass between zones).

06 // CRYPTOGRAPHIC ASSURANCE

Security Engineering & Verifiable Trust Architecture

How SafeArena uses SHA-256 hash chaining to detect tampering, and our defensible threat boundaries.

SHA-256 HASH CHAINING

Mathematical Integrity Mechanics

Append-Only JSONL: Every decision appends a canonical JSON line to data/audit_log.jsonl.

Canonical Serialization: json.dumps with sort_keys=True and ensure_ascii=True eliminates encoding ambiguity.

Cryptographic Linking: Record N contains prev_hash = SHA256(Record N-1). First record anchors to GENESIS.

Head Checkpoint: Latest count and head hash written atomically to _head.json.

Tail Protection: Deleting lines from the end of the log triggers a checkpoint count mismatch during verification.

TAMPER DETECTION PROOF

Concrete Tamper Detection

Tamper Scenario: Adversary changes 1 character in Record #2 (e.g. BLOCKED → ALLOWED).

Immediate Impact: SHA-256 digest of Record #2 changes completely.

Chain Break: Record #3's prev_hash points to old hash #2. Mismatch detected!

7 Automated Tamper Tests: Validates against: Edit, Edit+Rehash, Middle Deletion, Reordering, Forged Insert, Tail Truncation, and Corrupt JSON lines.

Verified Outcome: 100% of simulated corruption attacks detected without modifying live log.

HONEST TRUST MODEL

Defensible Threat Boundaries

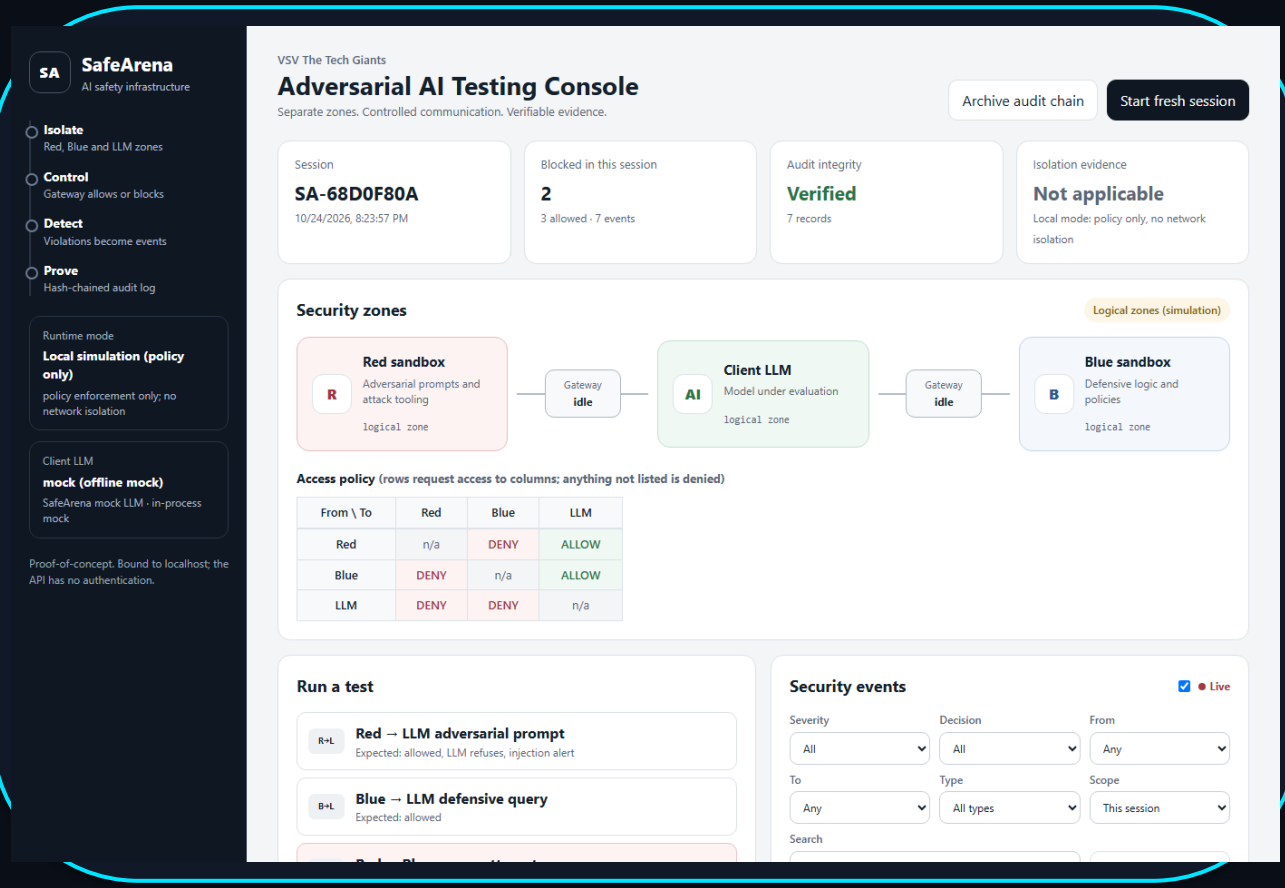
- No Exaggerated Claims: We do not claim SafeArena is unhackable or a silver bullet.
- Documented Limitation: A local hash chain catches in-flight edits, reordering, and truncations. However, an attacker with full host OS root access could rewrite both the log and checkpoint.
- Production Roadmap for Root Protection:

1. External HMAC key stored in KMS / HSM.
2. WORM (Write-Once-Read-Many) S3 Object Lock storage.
3. Periodically publishing head hashes to an external public transparency log or blockchain.

07 // OPERATOR DASHBOARD & EVIDENCE

The SafeArena Console: Live Telemetry & Verification

A zero-dependency, high-visibility operator interface built with strict Content Security Policy and live verification.



OPERATOR CAPABILITIES

1. Live Zone Status & One-Click Scenarios

- Interactive buttons trigger authorized Red→LLM queries, Blue defensive prompts, and unauthorized Red→Blue attempts.
- Gateway visual indicator switches dynamically from idle to active / blocked (turns red on 403 violations).

2. Filterable Security Event Stream

- Real-time event feed powered by SQLite.
- Filter dynamically by severity (info, medium, high, critical), decision (ALLOWED, BLOCKED), zone, or text query.

3. Instant Audit Chain Verification

- Single-click 'Verify audit chain' button recomputes every SHA-256 hash across the ledger in milliseconds.
- Displays verified head hash and total audit record count.

4. 7-Scenario Automated Tamper Test

- Push-button self-test copies the log into a tempdir and corrupts copies 7 ways; asserts the verifier catches every single one.

VERIFIED TEST EVIDENCE: 103 Automated Tests Passing · Playwright UI Smoke Test Passed · Zero Inline Scripts (XSS Hardened) · Parameterised SQL

08 // TECHNICAL IMPLEMENTATION

SafeArena Technology Stack & Engineering Status

Grounded in clean, hardened primitives with an honest distinction between implemented features and future roadmap.

GATEWAY & BACKEND

IMPLEMENTED & VERIFIED

Python 3.12 / Flask 3.1

- Modular Core: Engine modules (policy.py, gateway.py, audit.py, events.py) are completely framework-agnostic.
- FastAPI Port-Ready: Original MVP envisioned in FastAPI; built in Flask for offline zero-dep testability; app.py rewrite ready.
- Security Controls: Host/Origin header validation, strict CSP, rate limiting on POSTs, parameterised queries.

ISOLATION & DOCKER

COMPOSE TOPOLOGY READY

Multi-Container Topology

- Docker Compose v2: 4 dedicated containers (gateway, red-service, blue-service, llm-service).
- Isolated Bridges: red_net, blue_net, llm_net configured with internal: true.
- Container Hardening: read_only: true, tmpfs: [/tmp], cap_drop: [ALL], security_opt: [no-new-privileges:true].
- Status: Local simulation verified; Docker topology verified via compose unit tests.

AUDIT & EVENT STORAGE

IMPLEMENTED & VERIFIED

SHA-256 & SQLite DB

- Python hashlib: SHA-256 for canonical JSON record hashing.
- JSONL Append-Only: High throughput log file with atomic checkpoint file (_head.json).
- SQLite 3 Events: events.db with LIKE-escaping, SQL-injection immunity, and closed connection safety on Windows.
- Zero-Loss Rotation: Old chains archived to data/archive/ with cryptographic head anchoring.

AI MODEL INTEGRATION

MOCK + GEMINI API READY

Client LLM Abstraction

- Deterministic Local Mock: Supports 3 evaluation modes: benign prompt, injection refusal, and canary token leak.
- Google Gemini API: Pluggable upstream integration via LLM_PROVIDER=gemini and GEMINI_API_KEY in .env.
- Zero-Leak Key Architecture: Keys reside strictly in gateway memory; never returned in API responses, never logged, never sent to zones.

TRANSPARENCY NOTE: Local mode demonstrates policy enforcement only. Multi-container network isolation is claimed when Docker mode runs.

09 // MARKET & TARGET PERSONAS

Target Users & Real-World AI Security Applications

Empowering security engineers, AI development teams, and compliance officers with controlled testing infrastructure.

AI RED TEAMS

Adversarial AI Testers

Core Challenge:

Need to test aggressive jailbreaks, recursive prompt injections, and autonomous agent exploits safely.

SafeArena Value:

Blast-proof isolated sandbox where payloads cannot escape into companion tools or corporate assets.

Primary Use Case:

Automated red-teaming, prompt injection fuzzing, and model guardrail stress-testing.

Outcome: Zero Staging Escapes

APPSEC & DEVSECOPS

Enterprise Security Teams

Core Challenge:

Ensuring multi-tenant LLM applications enforce strict boundaries and do not leak internal state.

SafeArena Value:

Automated security gate in CI/CD that validates default-deny policies before production release.

Primary Use Case:

Pre-deployment policy validation for multi-agent systems and enterprise RAG pipelines.

Outcome: Automated CI/CD Gates

AI GRC & AUDITORS

Governance & Compliance

Core Challenge:

Satisfying emerging AI safety regulations (EU AI Act, NIST AI RMF, ISO 42001) with verifiable proof.

SafeArena Value:

Tamper-evident SHA-256 audit ledger proves exactly what prompts were tested, allowed, or blocked.

Primary Use Case:

Evidentiary documentation for third-party security audits and regulatory compliance filings.

Outcome: Non-Repudiable Evidence

ACADEMIC & RANGES

Education & Training

Core Challenge:

Teaching AI adversarial security and prompt injection defense in a safe, repeatable laboratory.

SafeArena Value:

Self-contained environment that students cannot break out of; immediate visual feedback on attack outcomes.

Primary Use Case:

Hands-on student AI capture-the-flag (CTF) challenges and defensive training.

Outcome: Safe Student CTF Lab

10 // INNOVATION & ROADMAP

Measured Innovation & Phased Engineering Roadmap

A pragmatic development roadmap progressing from our verified prototype to enterprise-grade trust infrastructure.

PHASE 1 — 100% COMPLETED

Core Prototype

Policy Engine: Framework-decoupled policy engine with strict default-deny enforcement.

SHA-256 Ledger: Hash-chained JSONL audit ledger with _head.json checkpointing.

Event Storage: SQLite persistent event store with multi-filter indexing.

Operator Console: Zero-dependency web console with 103 unit and integration tests passing.

Status: Production-Grade PoC

PHASE 2 — DOCKER READY

Container Isolation

Network Subnets: Multi-container Docker Compose with internal bridge networks (red, blue, llm).

Container Security: Hardened containers: read-only root, dropped caps, no-new-privileges.

Isolation Script: Host-side automated connectivity probe script (docker_isolation_test.py).

Mode Detection: Local simulation vs Docker runtime indicator on dashboard.

Status: Container Verified

PHASE 3 — NEXT MILESTONE

Cryptographic Identity

mTLS / Signed JWTs: Mutual TLS or signed JWT tokens for cryptographic zone attestation.

Identity Provenance: Eliminate caller-asserted identity; enforce cryptographic provenance per zone.

Dynamic DSL: Dynamic YAML policy DSL with hot-reloading rules.

Heuristic Models: Advanced semantic prompt-injection classifiers.

Status: Next Sprint

PHASE 4 — FUTURE VISION

Enterprise Trust Scale

Hardware Keys: Hardware Security Module (HSM) / AWS KMS integration for audit log signing.

WORM Storage: Write-Once-Read-Many cloud storage integration (AWS S3 Object Lock).

Transparency Log: Public transparency log publishing for non-repudiable verification.

Multi-Node Swarms: Distributed multi-node testbed for multi-agent swarm evaluations.

Status: Enterprise Scale

SAFEARENA CONCLUSION

HACK IN HILLS '26

“Test AI systems in a controlled environment. Enforce the rules. Record what happens. Verify the evidence.”

ISOLATE

Segregated Sandboxes

Red, Blue, and LLM zones run in separate network subnets with dropped capabilities.

CONTROL

Default-Deny Gateway

Every request is evaluated; unauthorized lateral movement blocked with 403 Forbidden.

DETECT

Real-Time Telemetry

Severity-rated events and injection heuristics committed to SQLite and operator UI.

PROVE

SHA-256 Audit Ledger

Mathematical proof of integrity; verified against 7 distinct tamper scenarios.

SafeArena · Created by VSV The Tech Giants · Hack in Hills '26

Working Codebase & Automated Test Suite Ready for Evaluation · Open Architecture · Python 3.12 / Docker

Thank you judges! We welcome your questions.